4-1 设置响应状态码

需求场景

收到客户端的请求后，服务端要响应，响应主要包含三个关键部分：响应状态码，响应头，响应体。

我们先看响应状态码，服务端如何设置响应状态码呢？

> 补充：对状态不清楚的复习【1.8 补充HTTP协议】，或者参考视频教程：https://www.51zxw.net/Show.aspx?cid=1003&id=117478

FastAPI的解决方式

FastAPI提供了一种简单便捷的方式设置，简单来说就是在路径装饰器中使用：`status_code`字段

示例1：直接给status_code字段一个状态码

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()


class User(BaseModel):
    username: str
    password: str


@app.post("/login", status_code=200)    # 直接指定一个响应码
def login(user: User):
    return {
        "username": user.username,
        "password": user.password
    }
```

示例2：status_code使用枚举值

> 补充：官方库http中的HTTPStatus是一个枚举类，枚举了所有的状态码

```
from http import HTTPStatus

from fastapi import FastAPI


app = FastAPI()


@app.post("/login", status_code=HTTPStatus.OK)  # 枚举值OK 其实就是数字200
def login():
    pass
```

示例3：status_code使用常量

> 从FastAPI导出的status，本质是直接使用starlette的status模块

```
from fastapi import FastAPI, status


app = FastAPI()


@app.post("/login", status_code=status.HTTP_200_OK)
def login():
    pass
```

4-2 设置响应头

需求场景

有的时候，在响应客户端请求的时候，需要在响应头中给客户端返回一些数据，此时该如何设置响应头呢？

FastAPI的解决方式

示例1：在路径函数中使用 `Response`

```
from fastapi import FastAPI, Response

app = FastAPI()


@app.post("/login")
def login(response: Response):
    response.headers["x-jwt-token"] = "this_is_jwt_token"
    return {"message": "Hello World"}
```

示例2：直接使用返回一个`Response`，比如使用FastAPI的默认响应方式JSONResponse

```python
from fastapi import FastAPI
from fastapi.responses import JSONResponse

app = FastAPI()


@app.post("/login")
def login():
    response = JSONResponse(
        content={"message": "Hello World"},
        headers={"x-jwt-token": "this_is_jwt_token"},
    )
    return response
```

扩展：使用返回Response的方式，也可以设置响应状态码，手动设置响应体，操作cookie等等

4-3 响应体数据

FastAPI响应数据时，默认响应是json的格式，具体使用的响应类是 `JSONResponse`

示例1：响应 普通结构数据

```python
from fastapi import FastAPI
from fastapi.responses import JSONResponse

app = FastAPI(default_response_class=JSONResponse)


@app.get("/")
def hello():
    return "hello world"
```

示例2：响应 列表嵌套字典的数据结构

```python
from fastapi import FastAPI
from fastapi.responses import JSONResponse

app = FastAPI(default_response_class=JSONResponse)


@app.get("/books")
def get_books():
    return [{"id": i, "name": f"图书{i}"} for i in range(1, 11)]
```

示例3：响应 基于Pydantic的模型类

```
from fastapi import FastAPI
from pydantic import BaseModel
from fastapi.responses import JSONResponse

app = FastAPI(default_response_class=JSONResponse)


class User(BaseModel):
    username: str
    password: str


@app.post("/login")
def login(user: User):
    return user
```

示例4：直接响应一个 `Response` 对象或 `Response` 的子类

```
from fastapi import FastAPI, Response
import json

app = FastAPI()


@app.get("/books")
def get_books():
    response = Response(
        content=json.dumps([{"id": i, "name": f"图书{i}"} for i in range(1,
11)]),
        media_type="application/json"
    )
    return response
```

4-4 使用响应模型response_model

需求场景

- 需求1：用户注册接口（需要用户名和密码），后端完成注册后一般会将用户信息返回，但此时不能返回用户密码，该如何操作？
- 需求2：想让接口返回的字段按照一定的规范来，有数据格式校验和转换等功能，该怎么办？
- 需求3：API文档展示响应字段和类型

FastAPI的解决方式

FastAPI提供了响应模型（`response_model`）的概念，就像定义模型类用来接收请求体数据，响应模型用来处理响应数据。

利用响应模型，可以实现返回字段的动态过滤，类型校验和类型转换等。

示例1：使用response_model控制过滤响应字段

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()


class User(BaseModel):
    username: str
    password: str
    email: str


class UserOut(BaseModel):
    username: str
    email: str


@app.post("/registe", response_model=UserOut)       # 指定了响应模型是UserOut，则只返回username和email两个字段
def registe(user: User):
    return user
```

示例2：封装示例1代码

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()


class BaseUser(BaseModel):
    username: str
    email: str


class UserIn(BaseUser):
    password: str


class UserOut(BaseUser):
    pass


@app.post("/registe", response_model=UserOut)
def registe(user: UserIn):
    return user
```

示例3：使用response_model做类型校验和类型转换

```
import typing

from fastapi import FastAPI

app = FastAPI()


@app.post("/demo", response_model=typing.Dict[str, int])
def demo():
    return {"code": 200, "value": "100"}        # 像一个数据的值，会自动转换为Int类型
```

4-5 用于过滤响应数据的五个参数

需求场景

很多时候，我们需要对响应模型中的某些字段做一个过滤操作，比如，

- 没有赋值的字段过滤掉
- 使用默认值的字段过滤掉
- 指定某些字段过滤掉
- 保留指定的字段等等

FastAPI的解决方式

FastAPI提供了5个配合响应模型一块使用的字段：

- response_model_include，只展示选中的字段
- response_model_exclude，选中的字段不展示
- response_model_exclude_unset，过滤掉没有赋值的字段
- response_model_exclude_defaults，过滤掉使用默认值的字段
- response_model_exclude_none，排序没有值的字段

示例：五个字段的基本使用

```
from typing import Union

from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()


class Item(BaseModel):
    name: str
    description: Union[str, None] = None
    price: float
    tax: float = 10.5


items = {
    "foo": {"name": "Foo", "price": 50.2},
```

```
    "bar": {"name": "Bar", "description": "The Bar fighters", "price": 62,
"tax": 20.2},
    "baz": {
        "name": "Baz",
        "description": "There goes my baz",
        "price": 50.2,
        "tax": 10.5,
    },
}


@app.get(
    "/items/{name}",
    response_model=Item,
    response_model_include={"name"},
    response_model_exclude={"tax"},
    response_model_exclude_unset=True,
    response_model_exclude_defaults=True,
    response_model_exclude_none=True,
)
def read_item_name(name: str):
    return items[name]
```

4-6 Pydantic模型类对象的dict方法

Pydantic模型类的对象会有一个 `dict()` 方法，调用该方法时会返回一个字典，字典的key是对象的每个字段，字典的值是对应字段的值。

示例1：使用模型类实例化一个对象，并调用dict()方式

```
from typing import Union

from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()


class Item(BaseModel):
    name: str
    description: Union[str, None] = None
    price: float
    tax: float = 10.5


@app.get("/item")
def get_item():
```

```
    item = Item(name="foo", description="desc", price=9.9, tax=1.2)
    return item.dict()      # 手动实例化对象item，  并调用dict()
```

示例2：dict()方法中的配置参数

```
def dict(
    self,
    *,
    include: Union['AbstractSetIntStr', 'MappingIntStrAny'] = None,
    exclude: Union['AbstractSetIntStr', 'MappingIntStrAny'] = None,
    by_alias: bool = False,
    skip_defaults: bool = None,
    exclude_unset: bool = False,
    exclude_defaults: bool = False,
    exclude_none: bool = False,
)
```

- response_model_include， 等效于：dict(include={})
- response_model_exclude，等效于：dict(exclude={})
- response_model_exclude_unset，等效于：dict(exclude_unset=True) 或 dict(skip_defaults={})
- response_model_exclude_defaults，等效于：dict(exclude_defaults=True)
- response_model_exclude_none，等效于：dict(exclude_none=True)

4-7 jsonable_encoder

需求场景

客户端提交的数据需要保存数据库，那此时就需要保证数据都是可序列化的，那如何保证待序列化的数据都是可以序列化的呢？

比如：

```
from datetime import datetime
import json
from typing import Optional

from fastapi import FastAPI
from pydantic import BaseModel


app = FastAPI()

class Item(BaseModel):
    title: str
    timestamp: datetime
    description: Optional[str] = None


@app.post("/item")
```

```
def create_item(item: Item):
    dd = item.dict()
    json.dumps(dd)          # 因为timestamp这个字段的类型是datetime，json识别不了，所以dumps时肯定会报错
    # save to db
    return item
```

FastAPI的解决方式

FastAPI提供了 `jsonable_encoder` 这个工具，它可以将一个python对象处理成一个字典，并保证字典中的每个值是可以被序列化的

示例1：`jsonable_encoder` 的基本使用，保证字典的值是可以被序列的

```
from datetime import datetime
from typing import Union
import json
from fastapi import FastAPI
from fastapi.encoders import jsonable_encoder
from pydantic import BaseModel

fake_db = {}


class Item(BaseModel):
    title: str
    timestamp: datetime
    description: Union[str, None] = None


app = FastAPI()


@app.post("/item")
def create_item(item: Item):
    json_compatible_item_data = jsonable_encoder(item)
    print(type(json_compatible_item_data))
    print(json_compatible_item_data)
    json.dumps(json_compatible_item_data)
    return item
```

示例2：`jsonable_encoder` 处理时也可以按条件过滤字段

```
from datetime import datetime
from typing import Union
import json
from fastapi import FastAPI
from fastapi.encoders import jsonable_encoder
from pydantic import BaseModel

fake_db = {}
```

```python
class Item(BaseModel):
    title: str
    timestamp: datetime
    description: Union[str, None] = None


app = FastAPI()


@app.post("/item")
def create_item(item: Item):
    print(item.dict())      # 字典中有description
    json_compatible_item_data = jsonable_encoder(item, exclude={"description"})
    print(json_compatible_item_data)    # 字典中没有description
    json.dumps(json_compatible_item_data)
    return item
```

4-8 响应部分总结

- 如何设置状态码：statuc_code、Response及其子类
- 如何设置响应头：Response及其子类
- 如何设置响应体：默认响应类是JSONResponse，也可以手动返回Response及其子类的对象
- 使用响应模型做字段过滤

- 关于不同类型的响应（下载文件等）在11章响应介绍